UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 06

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: Framework JS: Eventos, Renderizado Condicional/Iterativo, Formularios, Routing y Consumo de APIs (Async/Await, Axios)
Modalidad: Presencial / Laboratorio
Fecha: 07.10.2026
Duración: 90 minutos

APELLIDOS Y NOMBRES: PARIONA RAMOS SAUL ANIBAL

OBJETIVO DE LA PRÁCTICA

Crear una aplicación frontend de página única (SPA) utilizando un framework JS (React), configurar el entorno con herramientas modernas (Vite + npm/pnpm), implementar arquitectura basada en componentes con JSX, transmitir datos mediante props y composición con children, aplicar estrategias de estilado (CSS Modules / Tailwind / inline) y validar el comportamiento de **Client-Side Rendering (CSR)** frente al ciclo de vida del DOM.

RECURSOS REQUERIDOS

- Node.js LTS (≥20.x) + npm/pnpm
- Visual Studio Code (Extensiones: ES7+ React Snippets, React Developer Tools, Error Lens, REST Client)
- Navegador moderno + DevTools (Network, Performance, Console)
- Proyecto base: Vite + React + react-router-dom + axios
- API de prueba: https://jsonplaceholder.typicode.com/users o https://dummyjson.com/products
- Documentación oficial: react.dev, reactrouter.com, axios-http.com
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Tiempo aproximado	Concepto Clave	Restricción/Nota Técnica
1	Configurar proyecto: npm create	5 min	Routing cliente, gestión de	No usar HashRouter salvo justificación

	vite@latest, instalar reactrouter-dom y axios. Crear estructura: src/pages/, src/components/, src/services/, src/hooks/		dependencias, arquitectura SPA	técnica. Preferir BrowserRouter
2	Implementar enrutamiento: <BrowserRouter>, <Routes>, <Route>, <NavLink>. Crear Home, List, Form, NotFound	8 min	Client-side routing, historial de navegación, lazy loading opcional	Verificar que la navegación no recargue la página. Usar activeClassName para feedback visual
3	Consumo de API con axios + async/await en useEffect. Manejar estados: loading, data, error	12 min	Promesas, try/catch, cleanup en useEffect, AbortController	Prohibido usar .then().catch() anidados. Usar async/await con manejo explícito de errores
4	Renderizado iterativo (.map() con key) y condicional (&&, ternario). Formulario controlado con useState	10 min	List rendering, key prop, controlled inputs, event handling (onChange, onSubmit)	Validar que cada key sea único y estable. Evitar index como key si la lista muta

5	Integrar componentes, validar flujo de datos, auditar red (DevTools) y	10 min	State synchronization, network waterfall, UX	Documentar en comentarios cómo se evita el renderizado
	estado de UI en transiciones		states, React DevTools	innecesario tras fetch

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

Herramienta	**Uso Permitido**	**Prompt Ejemplo**	**Uso Prohibido**
ChatGPT/Claude/Copilot	Depurar useEffect dependencies, explicar AbortController, optimizar manejo de errores en axios	"¿Por qué miuseEffectse ejecuta dos veces en desarrollo y cómo evitar duplicar peticiones a la API?"	"Genera la SPA completa con routing, formularios y consumo de API"
React DevTools / Network Tab	Inspeccionar estado, verificar peticiones duplicadas, medir tiempo de respuesta	(Automático)	Modificar estado manualmente para "saltar" lógica de carga/error
JSON Schema / Mock API	Validar estructura de respuesta, simular errores 4xx/5xx	(Oficial)	Ignorar manejo de errores de red o fallbacks UI

Regla de laboratorio: El 75% de la estructura de componentes y transmisión de datos debe escribirse manualmente. Cada uso de IA para resolver conflictos de estilos o flujo de props debe llevar comentario // IA: [problema] → Solución manual: [explicación]

Evidencias del Ejercicio Desarrollado:

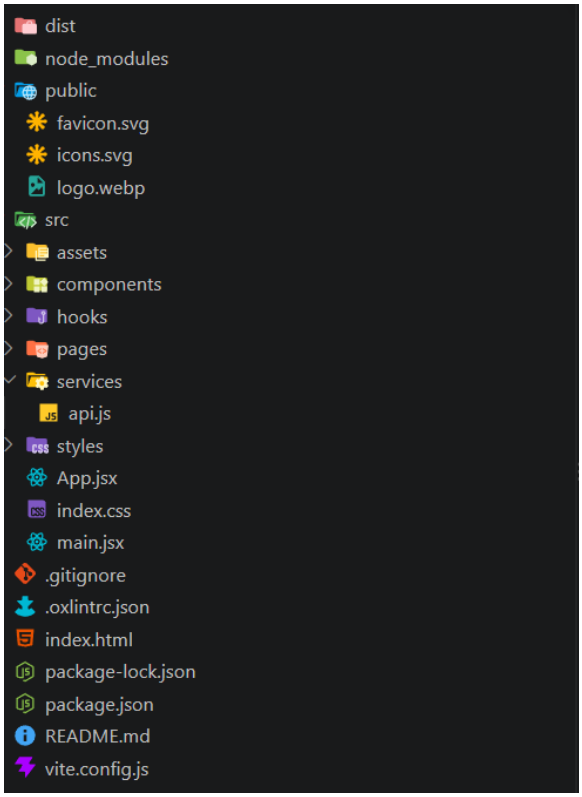
- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.
  1. Configuración del proyecto

```
npm create vite@latest semana6_aguilar
cd semana6_aguilar
npm install
npm install react-router-dom axios lucide-react
```

Dependencias principales:

- React y React DOM.
- Vite.
- react-router-dom.
- axios.
- lucide-react.
- CSS Modules.

ESTRUCTURA:



```
src/
├── components/
│   ├── Footer.jsx
│   ├── Navbar.jsx
│   ├── NetworkAudit.jsx
│   ├── ProductCard.jsx
│   └── Section.jsx
├── hooks/
│   ├── useProducts.js
│   └── useTheme.js
├── pages/
│   ├── Form.jsx
│   ├── Home.jsx
│   ├── List.jsx
│   └── NotFound.jsx
├── services/
│   └── api.js
├── styles/
│   └── ui.module.css
├── App.jsx
├── index.css
└── main.jsx
```

2. Enrutamiento SPA
   Se utiliza BrowserRouter, no HashRouter, porque la aplicación usa rutas limpias y navegación del lado del cliente:

```
<BrowserRouter>
  <App />
</BrowserRouter>
```

   En App.jsx se definen:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/catalogo" element={<List />} />
  <Route path="/pedido" element={<Form />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

   El componente Navbar utiliza NavLink. En React Router actual, activeClassName fue reemplazado por una función basada en isActive:

```
className={({ isActive }) =>
  isActive ? `${s.link} ${s.active}` : s.link
}
```

3. Consumo de API con Axios
   El consumo se implementa en src/services/api.js y src/hooks/useProducts.js. getProducts usa Axios y async/await:

```
const { data } = await http.get('/recipes', {
  params: {
    limit: 0,
    select: 'name,image,rating,prepTimeMinutes,tags,mealType',
  },
  signal,
})
```

Los productos de DummyJSON clasificados como postres se combinan con productos adicionales de pastelería para ampliar el catálogo.

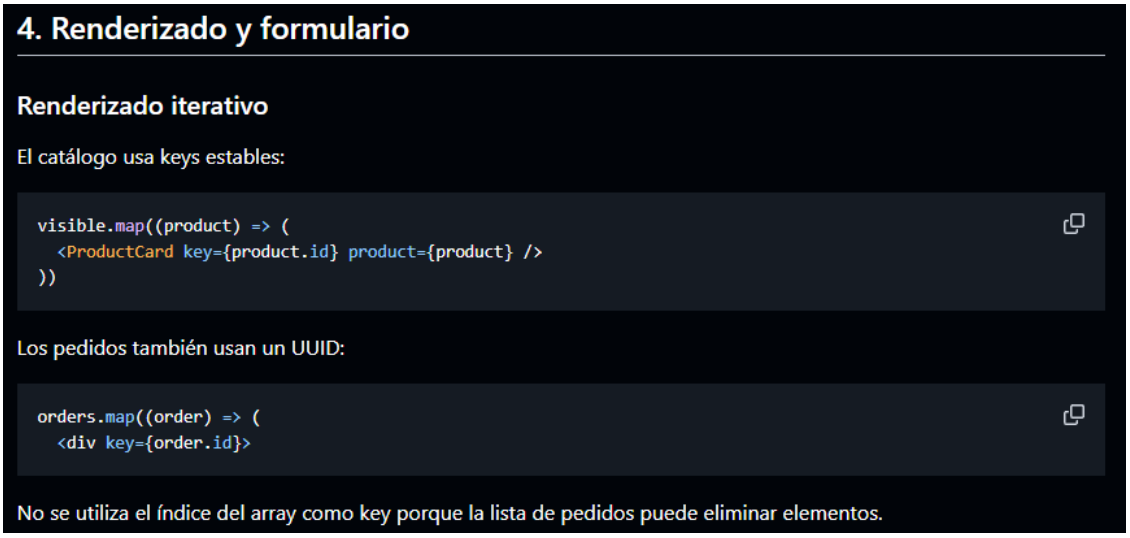
El hook mantiene un único objeto de estado:

```
{
  loading: true,
  data: [],
  error: null
}
```

AbortController

Cada efecto crea un controlador:

4. Renderizado y formulario



5. Integración, optimización y auditoría

Sincronización de estado

`loading`, `data` y `error` se actualizan con una sola llamada:

```
setState({
  loading: false,
  data: products,
  error: null,
})
```

Esto evita tres actualizaciones independientes y renders intermedios.

El filtrado se deriva durante el render:

```
const visible = data.filter(...)
```

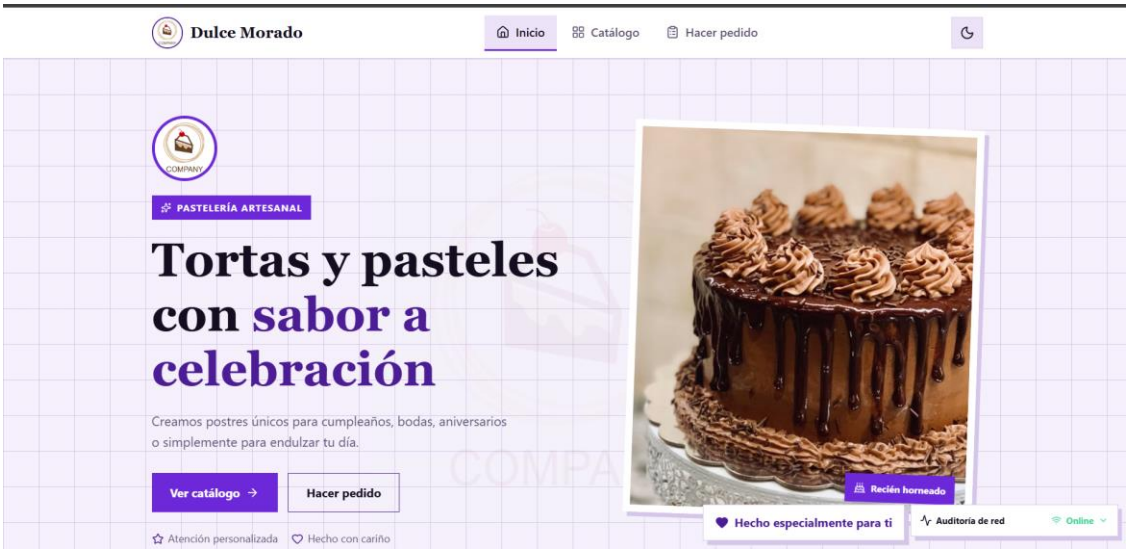
No se mantiene una segunda copia de los productos filtrados, por lo que escribir en el buscador no hace otro `fetch`.

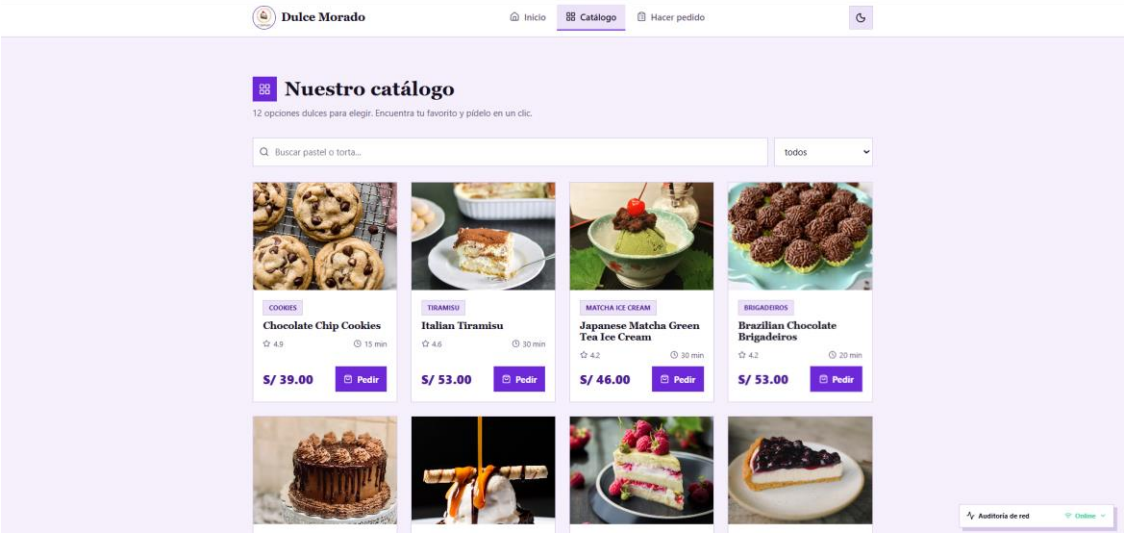
Los comentarios de `useProducts.js` y `List.jsx` documentan esta decisión.

Panel flotante de auditoría

`NetworkAudit.jsx` muestra:

- Añada capturas de la Página Web implementada de manera local





- Proporcione el enlace de Github de su proyecto

  https://github.com/saulPariona/SEMANA6_AGUILAR.git